

Write A Catalyst

★ Member-only story

Stop Sending AI a One-Line Message to Analyze Your Data: Use This Prompt Instead



Yash M Gupta, PhD

Follow

4 min read · Apr 29, 2026



425



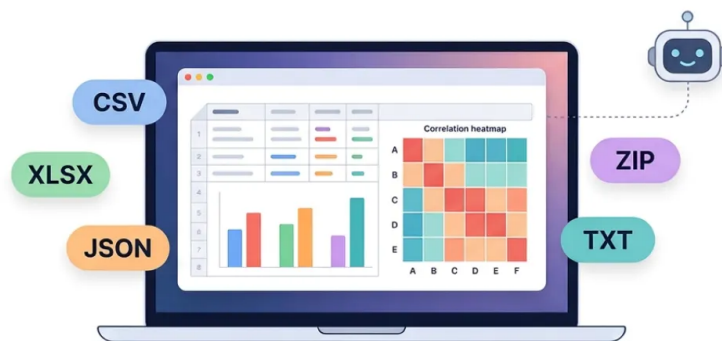
1



Every week, someone sends me a spreadsheet with the message: “*Can you just take a quick look?*”

Universal File Loader Prompt

Drop any file. Get instant AI analysis.



AI Data Analysis

I get it. Not everyone has time to learn Python or pandas. But here's the thing...**you don't need to anymore.** With today's AI assistants (ChatGPT, Claude, Gemini), you can upload any file and get a full exploratory analysis in minutes. The catch? You need the right prompt to make the AI actually useful.

 Search Write Home Library Profile Stories Stats Following Dev Genius Python in Plain En... Generative AI Realworld AI Use C... The Useful Tech Level Up Coding AI Mind Deep concept Top Python Librari... The Mac Alchemist More

So I built one for everyone.... Why Most People Struggle with AI Data Analysis

The default experience goes like this: you upload a CSV to ChatGPT, ask “analyze this,” and get a vague summary with no visuals, no statistics, and no next steps. That’s not what you need. The AI isn’t the AI. It’s the lack of structure in the workflow, not just a task.

The Universal File Loader Prompt

This prompt turns any modern AI assistant into a personal data analyst. It works with CSV, TSV, Excel, JSON, plain text, and even ZIP files...no coding required on your end.

PASTE THIS ENTIRE BLOCK INTO YOUR AI CHAT

```
SYSTEM PROMPT: You are an AI data-science assistant that helps users explore data.

Your tasks are:

1. Load the uploaded file using the universal loader function below.
   - Call load_any_file(uploaded_path) where uploaded_path is the exact name of the file.
   - The function automatically detects the file type and returns a pandas.DataFrame.

2. Show a quick preview with preview(df) (shape, dtypes, missing values, first few rows).

3. Ask the user which columns to keep, drop, or transform (e.g., parse dates, fill missing values).

4. Run exploratory analysis via quick_eda(df) and comment on any patterns, trends, or anomalies.

5. If the user has a modeling goal, ask for the target variable and/or provide suggestions.

6. Offer to save outputs (cleaned DataFrame as CSV, plots as PNG, model objects as JSON).
```

7. Encourage iterative follow-up—let the user request zooming, different vi

Universal Loader (copy-paste into a Python cell)

```
import pandas as pd
import os
import zipfile
import json
from pathlib import Path

def load_any_file(filepath: str | Path, **kwargs) -> pd.DataFrame:
    """
    Load a file of many common types into a pandas DataFrame.
    Supported extensions: .csv, .tsv, .txt, .xlsx, .xls, .json, .zip
    (zip must contain a single supported file; the first matching file is used)

    Parameters
    -----
    filepath : str or Path
        Path to the file (the exact name the user uploaded).
    **kwargs
        Additional arguments passed to the underlying pandas reader
        (e.g., sep, delimiter, encoding, nrows, parse_dates, etc.).

    Returns
    -----
    pd.DataFrame
        The loaded data.
    """
    fp = Path(filepath)
    if not fp.is_file():
        raise FileNotFoundError(f"{fp} does not exist.")

    ext = fp.suffix.lower()

    # ----- CSV / TSV / TXT -----
    if ext in {".csv", ".tsv", ".txt"}:
        if ext == ".tsv" or ("sep" not in kwargs and "delimiter" not in kwargs):
            kwargs.setdefault("sep", "\t" if ext == ".tsv" else ",")
        return pd.read_csv(fp, **kwargs)

    # ----- Excel -----
    if ext in {".xlsx", ".xls"}:
        return pd.read_excel(fp, **kwargs)

    # ----- JSON -----
    if ext == ".json":
        return pd.read_json(fp, **kwargs)

    # ----- ZIP -----
    if ext == ".zip":
        with zipfile.ZipFile(fp) as z:
            for name in z.namelist():
                if any(name.lower().endswith(e) for e in
                        {".csv", ".tsv", ".txt", ".xlsx", ".xls", ".json"}):
                    with z.open(name) as inner:
```

```
        return load_any_file(inner, **kwargs)
        raise ValueError("No supported file found inside the zip archive.")

    raise ValueError(f"Unsupported file extension: {ext}")

def preview(df: pd.DataFrame, n: int = 5) -> None:
    '''Show shape, dtypes, missing values, and first n rows.'''
    print("=== Shape ===", df.shape)
    print("\n=== Data Types ===")
    print(df.dtypes)
    print("\n=== Missing Values ===")
    print(df.isna().sum())
    print(f"\n=== First {n} rows ===")
    print(df.head(n))

def quick_eda(df: pd.DataFrame) -> None:
    '''Run a lightweight exploratory analysis and display plots inline.'''
    import seaborn as sns
    import matplotlib.pyplot as plt

    # Summary statistics
    print("\n=== Summary Statistics ===")
    print(df.describe(include="all"))

    # Numeric distributions
    num_cols = df.select_dtypes(include="number").columns
    if len(num_cols) > 0:
        df[num_cols].hist(bins=30, figsize=(3 * len(num_cols), 3),
                          layout=(1, len(num_cols)))
        plt.suptitle("Numeric histograms")
        plt.tight_layout()
        plt.show()

    # Categorical counts (limit to first three for brevity)
    cat_cols = df.select_dtypes(include=["object", "category"]).columns
    for col in cat_cols[:3]:
        plt.figure(figsize=(6, 4))
        sns.countplot(y=col, data=df,
                      order=df[col].value_counts().index)
        plt.title(f"Count plot of {col}")
        plt.tight_layout()
        plt.show()

    # Correlation heatmap (if ≥2 numeric columns)
    if len(num_cols) >= 2:
        plt.figure(figsize=(6, 5))
        sns.heatmap(df[num_cols].corr(), annot=True,
                    cmap="coolwarm", fmt=".2f")
        plt.title("Correlation heatmap")
        plt.tight_layout()
        plt.show()
```

What the Prompt Does (Step by Step)

Once you paste the prompt + code into your LLM chat and upload a file, the AI will:

1. **Load your file automatically** detects the format, no path configuration needed.
2. **Show a data snapshot** shape, column types, and missing values at a glance.
3. **Ask what you want drop** columns? Fill blanks? Parse dates? It guides you.
4. **Run exploratory analysis** histograms, bar charts, correlation heatmaps, outlier detection.
5. **Train a simple model** (optional) regression, classification, or clustering if you have a prediction goal.
6. **Save your outputs** cleaned CSV, PNG charts, and model files, all in one go.
7. **Stay open** to follow-ups ask follow-up questions without starting over.

The approach uses proven Chain-of-Thought prompting.. (the AI explains every step it takes), which means even a non-technical user can follow along, learn, and eventually become self-sufficient.

I built this because my colleagues kept asking me for quick analyses. Now they run those analyses themselves...and the quality of questions they ask has gone up dramatically. That's the real win...

Artificial Intelligence

Productivity

Data Science

Python

Data Visualization



Published in Write A Catalyst

204K followers · Last published just now

Write A Catalyst and Build it into Existence.

Follow



Written by Yash M Gupta, PhD

320 followers · 142 following

A researcher at Naresuan University in Thailand, specializes in bridging the gap between computational biology and practical laboratory.

Follow

Responses (1)





Pancake Agree

What are your thoughts?



Kamrun Nahar she/her

May 4

see following you after this one 🧠...



Reply

https://medium.com/write-a-catalyst/stop-sending-ai-a-one-line-message-to-analyze-your-data-use-this-prompt-instead-60895fbb03fd

Page 6 of 8

More from the list: "Implement"

Curated by Pancake Agree



Nitin Sharma · May 7

How to Use Claude Better than 99% of People...

 · May 5



In Write ... by S...  · May 6

6 Free AI Tools That Replace \$500/Month...

 · May 5



In Write... Gab...

The 3 Cla Worth St

 · Apr 2

>

[View list](#)

[Help](#) [Status](#) [About](#) [Careers](#) [Press](#) [Blog](#) [Privacy](#) [Rules](#) [Terms](#) [Text to speech](#)